

路演 15 MIN · 面试 45 MIN · 叙事作战手册

AI 小说转剧本 差异化叙事手册

不比"做了什么功能",比"解决了什么问题"

功能是入场券,人人都有;真正拉开差距的是 —— 你能否把一个普通课题,讲成一段"发现瓶颈 → 在约束下取舍 → 拿出可量化结果"的工程叙事。本手册把我们已经做出的三类硬优化,翻译成评委一听就记住的故事。

内部备战文档 · 供全员对齐话术与分工

目录

建议:路演组重点读 四 / 六 / 七,主讲与答辩组通读全篇。

- 一 核心叙事主张 —— 我们到底跟别人比什么
- 二 话术对照表 —— 把"功能"翻译成"解决问题"
- 三 五个招牌故事(STAR 结构,面试主线)
- 四 15 分钟路演逐分钟脚本
- 五 45 分钟面试作战地图与高频追问
- 六 亮点速记卡(临场前 5 分钟扫一眼)
- 七 Demo 防翻车清单
- 八 叙事红线 Dos & Don'ts
- 九 叙事优先级排序(评委视角)—— 时间不够时先说什么 ★

一句话总纲

"我们没有只做一个能跑的 demo,我们做的是**一个在真实约束下被打磨过的系统** —— 它更快、演示时不会翻车、并且把抽象的故事结构变成了你能用手转动的 3D 故事线。"

一、核心叙事主张

先定框:评委 15 分钟里能记住的东西极少,我们要替他们决定"记住哪三点"。

评委的心智模型

大多数组的叙事是"输入小说 → 调用大模型 → 输出剧本,我们做完了"。这在评委听来是同质化的 —— 功能谁都能拼出来。能拿分的,是让评委感到"这组真的踩过坑、做过判断、有工程深度"。

我们的差异化三支柱

支柱	一句话主张	证据(可现场指)
性能 瓶颈狙击式提速	整条链路是"阻塞式 LLM 调用链",我们定位到真瓶颈并做了 有界并行 ,而不是无脑加机器。	线上实测场景阶段 3.61x ;端到端建模 2.47x(222s→90s)
稳定 演示级可靠性	单点失败不拖垮全局、断连不丢结果、模型抽风有规则兜底 —— 为"现场不翻车"专门做工程。	蒙特卡洛实测:整本成功率 60%→98% (5%失败率);不崩率 100%
独特 3D 故事生产地图	别人给一张表,我们给一张 能转动、随生成生长 的故事线星图,还解决了节点重叠的布局难题。	3D 地图实时演示 + 去碰撞布局算法

三支柱站在同一个**异步任务架构底座**上 —— 提交即返回、消息队列保证可靠执行、SSE 实时回推进度。它让"快"被用户感知到、让"稳"在崩溃后仍能恢复。这是**招牌故事 5**,15 分钟里点到为止、45 分钟里展开讲。

贯穿全程的叙事公式

问题 → 约束 → 取舍 → 决策 → 可量化结果

永远不要停在"我实现了 X 功能"。每个亮点都要补足:它解决了什么痛?在什么约束下(演示稳定/不改模型输出/时间紧)?我们放弃了什么、选择了什么?结果好在哪、怎么量化?

金句 "评委记不住功能列表,但会记住一个'他们发现瓶颈、做了取舍、拿出数字'的故事。我们今天要讲的就是这三个故事。"

二、话术对照表

同一件事,左边是"完成功能"的平庸说法,右边是"解决问题"的升级说法。背下右列。

✖ 平庸说法(功能视角)	✔ 升级说法(问题/工程视角)
"我们用线程池让生成变快了。"	"我们先定位到瓶颈:整条链路是 串行的阻塞式大模型调用 ,总耗时≈调用次数×单次时延。于是把三个相互独立的阶段做了 有界并行 ,并刻意保留了大纲阶段的串行 —— 因为它有上下文依赖,强行并行会损害质量。"
"我们做了流式输出。"	"我们区分了 感知延迟 和 真实墙钟 :流式治的是前者,并行治的是后者,两者我们都做了。而且流式不是假的打字机回放 —— 是真实边生成、边落库。"
"我们做了异常处理。"	"我们为'现场演示'专门做了容错:单个场景生成失败会写 失败占位 而不是中断整本书;客户端断连后端仍会 跑完并保存 ;模型抽风时有 规则兜底 。目标是路演时不会因为一个偶发错误全盘崩掉。"
"我们做了一个可视化页面。"	"我们把抽象的故事结构做成可交互 3D 故事线,并解决了一个真实的 布局难题 :短篇里几乎每个角色都关联到所有场景,坐标会塌缩到一个点、标签糊成一团。我们用 去碰撞分道 + 焦点上下文标签 解决了。"
"后面的场景也有来源事件。"	"来源事件是 全书抽取 的、不是只取前几章 —— 后面的场景不是编的,是被真实事件锚定的。我们还做了事件归一化重编号保证溯源一致。"
"我们用了消息队列 MQ。"	"分析整本书是 分钟级长任务 ,远超请求超时。我们把'提交'和'执行' 解耦 :点一下立刻投递 MQ 返回 jobId、毫秒级响应,不占 Web 线程; 持久化队列 保证进程崩了也能由消息重建任务继续跑;同一项目进行中的任务 幂等去重 ,重复点击不会重复打爆模型。不是为了用 MQ 而用 —— 是长任务架构的必然。"

用法:任何评委问"你们做了什么",先用一句价值主张,再立刻切到右列的"问题—取舍—结果"结构。

三、五个招牌故事

面试主线。每个故事 = 背景 / 冲突 / 动作 / 结果 + 金句 + 预备追问。讲到能脱稿。

故事 1 · 瓶颈狙击:串行 LLM 流水线 → 有界并行

背景:整本书的处理是一条流水线 —— 切章 → 章节摘要 → 实体/事件抽取 → 场景大纲 → 逐场景剧本,每一步都是大模型调用。

冲突:一开始全是串行,整本书要等"调用次数 × 单次时延",几分钟起步,路演根本等不起。

动作:我们做了瓶颈建模: $墙钟 \approx 单次时延 \times 关键路径上的串行波次$ 。据此把三个相互独立的阶段并行化 —— 章节摘要、实体/事件抽取、逐场景剧本,各用有界线程池(并发 4 + 队列上限 + 拒绝回退到调用方),把对模型的并发压在可控范围。**关键判断:**大纲阶段我们故意不并行 —— 每个事件批次都要拿前面已生成的场景做上下文、按顺序编号,强行并行会破坏叙事连贯。

结果:把可并行阶段的耗时从"逐个相加"压到接近"单批耗时 × 批次数";并且我们知道剩下不可压缩的硬瓶颈在哪、为什么。

金句 "我们不是无脑加并发,而是分清了'哪里能并行、哪里因为上下文依赖必须串行' —— 后者才是真正考验判断的地方。"

预备追问 · "为什么不全部并行?" → 大纲的顺序上下文依赖。"并发设多少?" → 受模型限流约束,有界 4 起步,可配置环境变量。"怎么证明变快?" → 后端每阶段都打 elapsedMs 日志,有前后对比。

故事 2 · 演示级可靠性:为"不翻车"做工程

背景:大模型是不稳定外部依赖 —— 偶发超时、格式异常、限流。路演现场一旦某一步崩,整本书就废了。

冲突:原本一个场景失败会抛异常、中断整条流水线;现场演示这是致命的。

动作:三层防护 —— ① **单点失败隔离:**某场景失败写入"失败占位"并继续生成其余场景,失败的还会在下次被重试;② **规则兜底:**模型抽风时退回规则生成,降级而非崩溃;③ **断连自愈:**客户端断开后端仍跑完并落库,前端再靠轮询补回漏掉的状态。

结果:演示可靠性从"任一步失败即全崩"变成"局部失败可见、可重试、不影响全局"。

金句 "我们把'演示不翻车'当成一个真实的工程目标来设计,而不是祈祷现场网络好。可以现场拨一下网线给你看。"

现场杀手锏:如果允许,演示时主动制造一次失败(或断连),展示系统自我恢复 —— 这比任何 PPT 都有说服力。

故事 3 · 真·流式 + 落库:同时治"感知"与"真实"延迟

背景:用户最直观的痛是"点了之后干等,内容啪一下全出来"。

冲突:很多做法是前端假打字机回放 —— 看着像流式,其实内容早生成完了,治标不治本,而且切换场景会重复"伪流式"。

动作:我们做**真实流式**:后端边调用模型边把 chunk 通过 SSE 推给前端,流完**解析并落库**,把已保存的正式 Scene 回传更新视图。同时强调:**流式只解决感知延迟**,真实墙钟靠故事 1 的并行解决 —— 我们两条线都治。

结果:用户在等待中看到内容真实生长;且"看到的"和"存下来的"是同一份,没有表里不一。

金句 "流式和并行治的是两种不同的延迟 —— 一个是'感觉快',一个是'真的快'。我们没有用前者掩盖后者。"

预备追问 · "流式和最终结果一致吗?" → 一致,流完即解析落库回传。"断流了怎么办?" → 后端照常完成保存,前端轮询补偿。

故事 4 · 3D 故事生产地图:独特表达 + 真实布局算法

背景:大多数组用表格/列表展示结果。我们想让评委**一眼看懂故事结构**,做了可交互 3D 故事线(章节 / 事件 / 角色地点 / Scene 主线四类节点 + 连线)。

冲突:朴素布局会塌缩 —— 短篇里几乎每个角色/地点都关联到所有场景,按"关联场景平均位置"定坐标会让它们**全挤到一个点**,标签糊成一团(我们现场遇到过)。

动作:① **去碰撞分道布局**:按锚点从左到右扫描,强制相邻节点最小间距、整体回中、奇偶错位,角色与地点各占独立横带;② **焦点+上下文标签**:默认只显主线序号+光点,悬停或选中场景时相关标签才浮出 —— 专业图可视化的降噪手法;③ **随生成生长**:场景按顺序点亮、配合骨架占位;④ 画布 memo 化,流式刷新时不重渲染整棵 Three.js 树,避免现场卡顿。

结果:从"信息重叠看不清"变成"结构清晰、可交互、随生成生长"的差异化看点。

金句 "可视化不是贴张图就行 —— 真正难的是布局。我们解决了一个具体的节点塌缩与标签重叠问题,这才是工程。"

故事 5 · 异步任务架构:把"提交"和"执行"解耦 —— MQ + SSE 闭环

背景:分析整本书(切章 → 摘要 → 抽取 → 大纲 → 剧本)是分钟级长任务,远超浏览器/网关的请求超时。如果同步处理,一个 Web 线程会被一个任务占住几分钟,超时一断任务就白跑。

冲突:我们要同时满足三个互相矛盾的诉求 —— 点一下要**立刻有反馈**、任务要**可靠跑完(进程崩了不丢)**、用户还要**实时看到进度**。同步阻塞式调用一个都满足不了。

动作:把"提交"和"执行"解耦成消息队列驱动的异步任务系统:

- ① **提交即返回:**点一下就把任务投递到 MQ、立刻回 jobId(QUEUED),HTTP 毫秒级返回,不占 Web 线程;
- ② **持久化队列 = 崩溃可恢复:**队列与交换机都是 durable;消费端发现内存任务态丢失(进程重启)时,直接用消息体重建任务状态继续执行;
- ③ **幂等去重:**同一项目已有进行中的任务,直接复用其 jobId —— 重复点击 / 并发提交不会重复投递、不会重复打爆模型;还有"剧本已全部生成则直接短路返回"避免空跑;
- ④ **MQ + SSE 闭环:**MQ 管"任务可靠地跑",SSE 管"用户实时看到跑到哪" —— 消费端每过一个阶段(每章资产就绪 / 每批大纲就绪 / 每个场景完成)就推一个进度事件,前端**不轮询**,星图随之实时生长;SSE 配 15 秒心跳 + 自动重连兜底;
- ⑤ **安全边界:**SSE 只推阶段、数量、状态,绝不推正文、Prompt、AI 原始响应或密钥。

结果:一次点击同时拿到"即时反馈 + 可靠执行 + 实时进度";这套架构天然可水平扩展(多加消费者实例即可),而 Web 层不变。它也是流式与断连自愈能成立的底座。

金句 "我们把'提交'和'执行'解耦了 —— MQ 保证任务可靠跑完、崩了能恢复,SSE 保证你实时看到它跑到哪。这不是为了用 MQ 而用 MQ,是分钟级长任务绕不开的架构。"

预备追问 · "为什么用 MQ,不直接开个线程跑?" → 线程态随进程消失、不可恢复、不能削峰扩展;持久化队列三者都解决。"进程崩了任务丢吗?" → 不丢,消息持久化,消费端用消息体重建状态续跑。"重复点击会重复执行吗?" → 不会,进行中任务幂等复用 jobId。"MQ 和 SSE 各管什么?" → MQ 管可靠执行,SSE 管实时回推,两者配成闭环。"还有别的工程化吗?" → 增量分析(只处理新增内容)、任务投递失败立即标 FAILED 不留假 QUEUED。

四、15 分钟路演逐分钟脚本

原则:前 90 秒必须抛出"痛点 + 我们的不同";Demo 用**短篇**跑(快、稳、故事线饱满)。

时间	你说什么(要点)	你演 / 强调
0—1′	钩子:"小说转剧本不难,难的是 又快、又不翻车、还能让人看懂故事结构 。这三点是我们和别人的区别。"	一句话定位,不进功能细节
1—3′	现场导入 短篇 ,点开始 —— 顺势点一句"提交瞬间就返回了,因为是 异步任务架构(MQ+SSE) ,后台可靠地跑、前台实时回推进度"。	启动 demo;镜头给到进度与 3D 地图
3—6′	讲故事 1(并行提速) :瓶颈建模 + 三阶段并行 + 大纲为何故意串行。亮 elapsedMs 前后数字。	切后端日志/对比图,给出真实数字
6—9′	讲故事 4(3D 地图) :点选场景看上下文高亮、悬停看节点、讲去碰撞布局解决了什么。	转动地图、点选、悬停现场交互
9—11′	讲故事 3(真流式) :重生成一个场景,看内容真实流式落库。点明"感知 vs 真实延迟我们都治"。	触发一次流式生成
11—13′	讲故事 2(可靠性) :制造一次失败/断连,展示失败占位+自愈。"为不翻车做工程"。	(可选)现场制造失败演示恢复
13—14′	收束:回到三支柱一句话各一句,呼应开场。	三支柱 slide 定格
14—15′	留 buffer / 主动抛一个我们最想被问的技术点引导提问。	引导评委问到你的强项

分工建议:1 名主讲控节奏与叙事,1 名操作 demo(不抢话),1 名盯后台/随时准备回答深问。Demo 与讲解**并行**,别让大家盯着进度条干等。

五、45 分钟面试作战地图

目标:把对话主动引向四个招牌故事,而不是被动答功能题。

开场设框(前 3 分钟自己主导)

主动说:"我们想重点讲三件我们觉得有工程含量的事 —— 提速的瓶颈分析、演示级的容错、以及 3D 故事线的布局难题。" —— **替评委设定话题地图**,后面所有问题都更容易落回你的强项。

高频追问与强答案

问题	强答案骨架
"这功能别人也能做,你们差异在哪?"	"功能是入场券。差异在于我们做了瓶颈分析后的 有界并行 、为现场可靠性做的 失败隔离/兜底/自愈 、以及解决了 3D 布局塌缩的 去碰撞算法 —— 这三点是判断力和工程,不是堆功能。"
"为什么不直接全部并行,越快越好?"	"大纲阶段有 顺序上下文依赖 ,每批要拿前文场景当上下文,强行并行会破坏连贯;而且模型有限流,无界并发会被拒。所以是 有界并行 + 保留必要串行。"
"怎么证明你们真的变快了?"	"两条证据:① 后端每阶段打 elapsedMs 日志,可现场看;② 我们写了 可复现基准 (与生产同语义的线程池),实测可并行阶段各 3.3—3.8×、 端到端 222s→90s≈2.47x ,并扫描出最佳并发度。数字能在本机重跑。"
"模型出错/超时怎么办?"	"三层:单点失败写占位不中断全局、规则兜底降级、断连后端跑完落库前端轮询补偿。"
"流式是不是前端假装的?"	"不是。后端真实边生成边推 SSE,流完解析落库,看到的与存下的一致。我们之前删掉了假打字机方案。"
"3D 地图技术难点?"	"布局塌缩与标签重叠。去碰撞分道 + 焦点上下文标签 + 画布 memo 防止流式期间重渲染。"
"为什么要引入 MQ?直接处理不行吗?"	"分析整本书是 分钟级长任务 ,同步会占满 Web 线程、超时即白跑。MQ 让我们 提交即返回 jobId , 持久化队列 保证崩溃可恢复, 幂等去重 防重复提交,还天然可水平扩展。"
"进程崩了/重启了任务会丢吗?"	"不会。队列与交换机持久化,消费端发现内存态丢失时用消息体 重建任务状态续跑 ,投递失败会立即标 FAILED,不会留下假 QUEUED。"
"MQ 和 SSE 各解决什么,为什么都要?"	" MQ 管可靠执行 , SSE 管实时回推 ,两者配成闭环:提交后不轮询,消费端每过一阶段推一个进度事件,星图实时生长;SSE 只推状态、不推正文/Prompt/密钥。"

"质量怎么保证不是瞎编?"	"事件全书抽取并归一化重编号,场景按 sourceRefs 溯源;还有校验报告和 YAML 导出显式标出失败/警告。"
"你个人具体做了什么?"	(每人准备一句)"我负责 X,遇到 Y 问题,做了 Z 取舍,结果 W。"

把"被动答题"变"主动叙事"的小技巧

- 每个回答结尾反挂一个钩子:"这块如果展开,还有一个我们做的取舍....." 引导下一个问题往你强项走。
- 遇到不熟的问题,诚实承认边界,再桥接回招牌故事:"这点我们没深做,但相邻的 X 我们处理过....."
- 数字要张口就来(并发度、阶段数、节点类别数、elapsedMs 量级)。

六、亮点速记卡

临场前 5 分钟扫这一页。每人手机里存一份。

三支柱(必须张口就来)

瓶颈狙击式有界并行

演示级容错(隔离/兜底/自愈)

3D 故事线 + 去碰撞布局

关键数字 / 事实(把测得的数字填进 [1])

- 流水线阶段:切章 → 摘要 → 实体/事件抽取 → 大纲 → 逐场景剧本(5 段,全是 LLM 调用)。
- 已并行阶段:章节摘要 / 实体事件抽取 / 逐场景剧本,有界并发默认 4、可配置。
- 故意保留串行:大纲(顺序上下文依赖,不可并行)。
- 提速量级(建模 + 线上真跑双验证):场景阶段线上实测 3.61x(98.7s→27.4s,9 个真实模型时延);端到端建模 2.47x;串行大纲占 33%(剩余硬瓶颈)。详见《优化量化报告》第七章。
- 线上实测铁证:异步提交响应 40—66ms(MQ);场景 9/9 PASSED、失败 0;并发 4 路(scene-script-1..4);布局重叠对 100→0。
- 地图节点类别:4 类(章节 / 事件 / 角色·地点 / Scene 主线)。
- 容错三层:失败占位 + 规则兜底 + 断连自愈。
- 异步架构:MQ 解耦提交/执行 + 持久化队列(崩溃可恢复) + 幂等去重 + SSE 实时回推(不轮询,15s 心跳自动重连)。
- 安全边界:SSE 只推阶段/数量/状态,不推正文/Prompt/AI 原始响应/密钥。
- 可靠性(蒙特卡洛实测):整本成功率 60%→98%(5% 单点失败率),不崩率 100%。
- 感知延迟(真流式):首屏 12s→0.7s ≈ 17x;墙钟靠并行、感知靠流式,两条线分治。
- 可扩展性:DB 连接占用 12s→0.2s ≈ 60x(慢调用移出事务),并发16/池10 排队 6→0。
- 成本控制(已落地实测):模型分级(摘要/抽取下放 flash、场景/大纲保 pro)→ 单本成本 降 ~31%、质量零退化,摘要阶段顺带 49s→7s;单本 ~40K token。
- 踩坑深度:11 个 fix 分支 = 11 个真实问题(长事务/连接泄漏/SSE 断连/脏 JSON...),166 commit、+12.7k 行。

金句(挑 2 句反复用)

金句 "功能是入场券,我们比的是发现瓶颈、做取舍、拿数字的工程能力。"

金句 "我们分清了能并行的和因上下文必须串行的 —— 后者才考验判断。"

金句 "流式治感知延迟,并行治真实延迟,我们两个都治。"

金句 "我们把'演示不翻车'当工程目标设计,而不是祈祷网络好。"

金句 "我们把提交和执行解耦:MQ 保证任务可靠跑完、崩了能恢复,SSE 保证你实时看到它跑到哪。"

千万别说(见第八章红线)

别说"就是调了个大模型 API"、别念功能清单、别在没数字时说"快很多"。

七、Demo 防翻车清单

路演前 30 分钟逐条打勾。现场稳定 > 内容炫酷。

项目	动作	状态
演示文本	用短篇(人物少、事件清晰、单一主线),阶段批次少→生成快、故事线又饱满。	☐
模型预热	开场前先跑一次丢弃请求,避免首调用冷启动慢一截。	☐
数据初始态	清到干净初始状态,避免脏数据导致展示异常。	☐
降级路径	确认模型失败时规则兜底可用;准备一份"已生成好的项目"作为 离线后备 。	☐
网络	确认 AI 接口可达;有备用网络;断连自愈现场可演示。	☐
并发参数	确认并发度设置合理,避免触发模型限流被拒。	☐
日志可见	准备好能展示 elapsedMs 的窗口/截图作为"提速证据"。	☐
地图交互	预先点选/悬停过一遍,确认标签浮出、上下文高亮正常。	☐
分工彩排	主讲/操作/答辩各就位,至少完整走一遍 15 分钟。	☐

兜底原则

现场任何一步卡住:**不要僵等**。立刻切到"我们正好讲讲这背后的设计"——把等待时间填成叙事;同时操作手切到离线后备项目继续演示。

八、叙事红线 Dos & Don'ts

最后对齐:这些是会扣分/加分的细节。

✓ Do

- 每个亮点都讲成"问题→约束→取舍→结果"。
- 主动设框,替评委决定记住哪三点。
- 用真实数字(elapsedMs、并发度、阶段数)。
- 承认边界,再桥接回强项。
- 强调判断力:哪里并行、哪里故意不并行。
- Demo 与讲解并行,不让评委盯进度条。
- 现场展示容错(制造一次失败再恢复)。

✗ Don't

- 念功能清单("我们还做了...还做了...")。
- 说"就是调了个大模型"自我矮化。
- 没有数字时说"快了很多/好了很多"。
- 把感知优化(流式)说成真实提速。
- 夸大成"全自动无错",被一个失败当场打脸。
- 四个人抢着答、互相打断。
- 在评委没问时陷入实现细节自嗨。

收尾:一段可以背下来的 60 秒电梯陈述

"小说转剧本的功能不稀奇,稀奇的是把它做快、做稳、做得能看懂。我们做了三件有工程含量的事:一,把串行的大模型流水线做了有界并行,并想清楚了大纲阶段为什么必须留串行;二,为现场演示做了失败隔离、规则兜底、断连自愈三层容错;三,把故事结构做成随生成生长的 3D 故事线,并解决了节点塌缩与标签重叠的布局难题。我们比的不是功能多,而是发现问题、做出取舍、拿出结果的能力。"

九、叙事优先级排序(评委视角)

评委 15 分钟能记住的就 2—3 件事。赢的不是讲得多,是**讲对顺序**。下面用评委视角给所有量化叙事打分排序。

评委到底在用什么打分(5 个维度)

维度	评委心里的潜台词
差异化	"这点别的组有没有?"—— 能不能跟同质化方案拉开
可验证	"是真的还是 PPT?"—— 能否现场证明 / 复现 / 拿日志
工程深度	"是堆功能还是有判断?"—— 取舍、瓶颈、为什么这么做
记忆点	"散会后我记得住吗?"—— 有没有画面感 / wow
易懂	"非专业评委听得懂吗?"—— 门槛越低越好

打分与排序(5 分制,总分 25)

叙事	差异	验证	深度	记忆	易懂	总分
S 可靠性可算成概率(整本成功率 60%→98%,拔网线)	5	5	4	5	4	23
S 3D 故事地图 + 去碰撞布局(100→0)	5	4	4	5	5	23
S 瓶颈狙击 + 大纲故意串行(实测 3.61x)	4	5	5	3	4	21
A 建模预测→线上真跑双向印证(元叙事)	5	5	5	3	3	21
A 踩坑地图(11 个 fix 分支)	4	4	5	3	4	20
B DB 连接占用 60x(长事务→拆分)	4	4	5	3	2	18
B MQ 异步架构 + 提交 66ms 实测	3	5	4	3	3	18
B 真流式 TTFB 17x(感知延迟)	2	4	3	4	5	18

三梯队 + 用在哪

第一梯队(开场就抛,必讲) —— ① **3D 地图**(demo 一开就是它,抓眼球) → ② **可靠性=可算的概率**(60%→98%,可现场拔网线,最高差异化) → ③ **瓶颈分析 + 大纲故意串行**(实测 3.61x,体现判断力)。三件事讲完,评委已经记住你们了。

第二梯队(深问时展开,技术评委吃这套) —— ④ **DB 连接 60x**("能不能上线扛并发") · ⑤ **MQ 异步 + 66ms**(解耦/崩溃恢复)。

第三梯队(点缀/收口) —— ⑥ **真流式 17x**(demo 顺口一提) · ⑦ **踩坑地图 11 fix**(收口,证明深度与投入)。

看人下菜:评委类型适配

评委类型	优先砸这几条
技术 / 架构评委	瓶颈+故意串行 → DB 连接 60x → MQ 崩溃恢复 → 双向印证方法论。深度越挖越加分。
业务 / 产品评委	3D 地图(看得见)→ 可靠性"拔网线不翻车" → 真流式"等待中内容在长"。讲体感,别堆术语。
混合 / 不确定	走第一梯队三件套:地图(易懂)+ 可靠性(差异)+ 判断力(深度),三个维度全覆盖。

如果只剩 30 秒(电梯版,背下来)

"我们做了三件别人没做的事:一,把**稳定性做成了可算的概率** —— 5% 故障率下整本成功率从 60% 提到 98%,可以现场拔网线;二,提速不是无脑加并发,我们**定位了瓶颈、还想清楚大纲为什么必须串行**,场景阶段线上实测 3.61x;三,把故事结构做成**能转动的 3D 星图**,还解决了节点塌缩的布局难题。而且每个数字我们都**建模预测过、也线上真跑验证过**。"

金句 "评委记不住功能,但会记住'稳定能算成概率、提速有判断、可视化有算法'——这三件事按这个顺序讲。"

唯一贯穿全程的红线:每抛一个数字,半句话补上"这是我们**真跑/真算出来的,可复现**"。可信度是把上面所有叙事黏在一起的胶水 —— 它本身不单独占一个名额,但每条都要靠它。